SpecFirst Phase 2 Architecture

Pattern Classification: turning Phase 1 component facts into a safe, explainable UI pattern decision

Purpose

Phase 2 answers one question: what kind of UI pattern is this React component?

Phase 1 produces facts about the component. Phase 2 interprets those facts and decides whether the pipeline can safely continue. For the current MVP, the target supported pattern is react-select-only-combobox.

Core rule: Phase 2 classifies the pattern. It does not load rules, generate tests, patch code, or claim accessibility compliance.

Input to Phase 2

The input is the Phase 1 artifact: componentAnalysis.json. In the current working example, the analyzer identified a named React TypeScript component with label, options, value, onChange, disabled, and className props; open state; a button trigger; a conditional ul popup; mapped li options; and no existing ARIA roles or keyboard handler.

Input field	What Phase 2 uses it for
component.name / component.path	Carries component identity into classification.json.
props	Detects collection, value, callback, label, boolean, and styling props.
state	Detects open-close state, selection state, or active-index state.
jsxElements	Checks whether native elements such as button, input, select, ul, li, a, form, or dialog exist.
interactionModel	Identifies candidate trigger, popup, option elements, and control state.
renderPatterns	Detects conditional popup, options rendered from map, selectable options, and trigger-controls-popup.
accessibilitySignals	Helps determine whether the component is already partially implemented or missing roles/states.
nativeElementSignals	Detects native select and whether a native select may be preferable.
eventStateLinks	Shows whether option clicks call onChange, close popup state, or trigger actions.

Output of Phase 2

Phase 2 writes classification.json in the current run folder. This file should be the only thing Phase 3 needs to decide which rule manifest to load.

Output section	Purpose
component	Repeats component name and path for traceability.
classification	Stores selected pattern, status, confidence, and confidence level.

evidence	Lists positive evidence that caused the selected pattern to win.
blockers	Lists hard reasons the chosen pattern cannot be used, if any.
rejectedPatterns	Explains why alternatives were rejected.
manualReviewNotes	Carries caution notes such as native select may be preferable.
nextPhase	Contains manifestId and canContinue. Phase 3 must obey this.

Expected Output for the Current Input

For the current SelectOnlyCombobox fixture, the expected classification is high confidence react-select-only-combobox. The component has an options collection, a value prop, an onChange callback, an open state, a button that toggles the popup, a conditional ul popup, mapped li options, and no input element.

Expected result: pattern = react-select-only-combobox; status = supported; confidence around 0.90 or higher; nextPhase.manifestId = react-select-only-combobox; nextPhase.canContinue = true.

How Phase 2 Works

Phase 2 should be deterministic and explainable. Do not use a model to make the final classification. Use a weighted scoring system plus hard blockers. Bob can later help explain or patch, but the Phase 2 decision should come from transparent rules.

#	Step	Direction
1	Load componentAnalysis.json	Read the Phase 1 artifact from the active run folder. Do not reread source unless needed for debugging.
2	Score candidate patterns	Compare the extracted facts against known pattern scorecards. For MVP, fully implement the select-only combobox scorecard.
3	Apply hard blockers	Reject a pattern if certain facts make it unsafe, such as a native select, text input, links, or nested interactive option content.
4	Calculate confidence	Convert score into a normalized confidence value and confidence level: high, medium, low, or very-low.
5	Reject alternatives	Explain why editable combobox, menu button, navigation menu, or native select were not chosen.
6	Set continuation decision	If supported and confident, set nextPhase.canContinue to true and provide the manifestId. Otherwise stop safely.

Supported Pattern for MVP

The only automatic continuation target for the MVP should be react-select-only-combobox. Other patterns may be detected or rejected, but they should not proceed to Phase 3 unless a corresponding manifest exists and has been validated.

Pattern Registry

Pattern	Status in MVP	Reason
react-select-only-combobox	supported	Main demo target and rule manifest exists.

react-editable-combobox	planned	Needs different keyboard and text-entry rules.
react-menu-button	planned or unsupported	Actions are different from value selection.
react-navigation-menu	unsupported	Navigation patterns should not receive combobox/listbox remediation.
native-select	manual-review	Native select is already a different implementation path.
unknown	unsupported	Not enough evidence to continue safely.

Select-Only Combobox Evidence

The classifier should award confidence when the component has signs of a custom single-select control. For the current fixture, most of these are present.

Evidence	Meaning	Strength
options collection prop	The component is driven by a list of choices.	strong
value prop	The component has a selected value.	medium
onChange callback	Selecting an option notifies the parent.	strong
open-close state	The component controls popup visibility.	strong
candidate trigger element	There is a control that opens the popup.	strong
candidate popup element	There is a rendered popup or list.	strong
mapped option elements	Options are rendered from a collection.	strong
selectable options	Option click calls a selection callback or updates value.	strong
no input element	This is select-only, not an editable combobox.	strong

Hard Blockers

Hard blockers prevent unsafe continuation. If a blocker fires, Phase 2 should not output supported, even if the score is high.

Blocker	Why it matters
Native select is present	Do not remediate a native select as a custom combobox.
Text input is present	This may be an editable combobox, which has different rules.
Anchors or hrefs inside options	This may be navigation, not value selection.
Nested buttons, checkboxes, or links inside option items	Listbox/option semantics may be incorrect for interactive content.
No selectable options detected	There is no clear select-only behavior to classify.
Multiple selection detected	Multi-select requires a different manifest.

Confidence and Status Model

The classifier should convert weighted evidence into a confidence score, then combine it with blocker checks to set status. A high score with no blockers can continue. A medium score should become ambiguous. A blocked or low-confidence result should stop.

Confidence	Suggested meaning	Pipeline behavior
0.85 - 1.00	High confidence	Continue if pattern is supported and no blockers exist.
0.65 - 0.84	Medium confidence	Usually mark ambiguous unless the project deliberately allows it.
0.45 - 0.64	Low confidence	Do not continue automatically.
0.00 - 0.44	Very low confidence	Unsupported or unknown.

Rejected Alternatives

The classifier should explain what it did not choose. This makes the decision auditable and protects the pipeline from applying the wrong rule manifest. For the current fixture, the expected rejected alternatives are editable combobox, menu button, navigation menu, and native select.

Rejected pattern	Expected reason for current input
react-editable-combobox	No input element or freeform text entry was detected.
react-menu-button	Items select values through onChange rather than trigger arbitrary actions.
react-navigation-menu	No links, hrefs, or routing targets were detected.
native-select	No native select element was detected, although native select may be preferable.

Handoff to Phase 3

Phase 3 should not re-decide the pattern. It should read nextPhase from classification.json. If canContinue is true, it loads the specified manifest. If canContinue is false, it stops and emits a safe report or manual review message.

For this input, Phase 2 should hand off manifestId = react-select-only-combobox and canContinue = true.

Failure Modes

Failure mode	Safe handling
Ambiguous component shape	Output status = ambiguous and stop before Phase 3.
Hard blocker present	Output status = unsupported and list blockers.
Unsupported but identifiable pattern	Record the likely pattern, explain unsupported status, and stop.
Low confidence	Do not force a manifest. Ask for manual pattern review.
Conflicting evidence	Prefer safety: mark ambiguous rather than continuing.

Definition of Done

• Reads componentAnalysis.json from the active run folder.

• Scores the select-only combobox pattern using clear evidence.

• Applies hard blockers before allowing continuation.

• Outputs confidence score and confidence level.

• Explains why alternatives were rejected.

• Includes manual review notes, especially native select preference when relevant.

• Writes classification.json.

• Sets nextPhase.manifestId and nextPhase.canContinue.

• Stops safely when ambiguous, unsupported, or blocked.

• Has fixtures for select-only combobox, native select, editable combobox, navigation menu, and action menu.

Summary

Phase 2 is the safety gate between static analysis and rule loading. It converts component facts into a pattern decision. For the current input, the correct output is a high-confidence supported classification as react-select-only-combobox. The classifier should be deterministic, weighted, explainable, and willing to stop when evidence is weak or unsafe.